

# PYTHON MASTER STUDY BOOK

A Complete Textbook for Programming with Python (Course Code  
2501IT42)

VOLUME 3  
UNIT III — Functions and Packages

This volume covers functions, scope, modules and packages, top-down design, and gives recursion the slow, careful, visual treatment it needs — including call-stack diagrams for every recursive example.

# UNIT III

## Functions and Packages

---

### Learning Objectives (CO3)

- Write functions with parameters, default values, keyword arguments, and return values.
- Understand local vs global scope, and why it matters.
- Use lambda functions for short, throwaway logic.
- Import and use modules and packages, including your own.
- Break a problem into smaller functions (top-down design).
- Understand recursion deeply: base case, recursive case, and the call stack.

### Prerequisites

Units I and II — especially loops, conditionals, and the core data types, since functions constantly use them internally.

## 1. Function Definition and Function Call

### LEVEL 1 — What is a function and why do we need it?

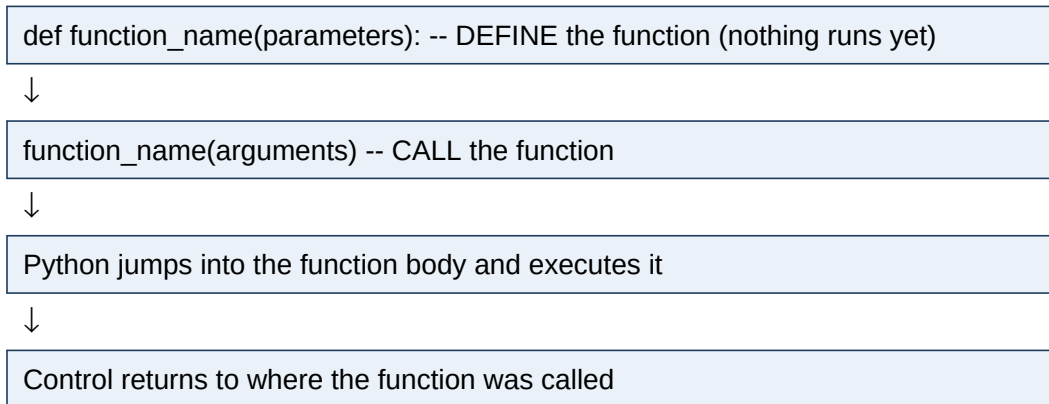
A **function** is a named, reusable block of code that performs a specific task. Instead of copying the same lines of code every time you need them, you write them once inside a function, give the function a name, and then simply **call** that name whenever you need that task done.

Think of a function like a coffee machine. You don't rebuild the whole brewing process by hand every time you want coffee. You press one button ("call the function"), and the machine internally runs all its steps and hands you the result.

```
def greet():                # def keyword starts a function DEFINITION
    # this is the function BODY -- must be indented
    print("Hello!")

greet()                     # this is a function CALL -- runs the body
greet()                     # calling again runs it again
```

*Definition vs Call -- what actually happens*



### COMPETITIVE EXAM / GATE FOCUS

A very common beginner misconception: **DEFINING** a function does not run its body. The code inside runs only when the function is **CALLED**. This is exactly like writing a recipe (definition) versus actually cooking it (call) -- writing the recipe down does not put food on the table.

## 2. Parameters and Arguments

A **parameter** is the name listed inside the parentheses in the function definition — it is a placeholder. An **argument** is the actual value you pass in when calling the function. People often use the words interchangeably in casual speech, but exams frequently test the precise distinction.

```
def greet(name):          # 'name' is a PARAMETER (placeholder)
    print("Hello, ", name)
```

```
greet("Asha")             # "Asha" is an ARGUMENT (actual value)
```

### MUST REMEMBER

- Parameter = the placeholder name in the function definition.
- Argument = the actual value supplied at the function call.

### Positional, Keyword, and Default Arguments

```
# city has a DEFAULT value
def student(name, age, city="Hyderabad"):
    print(name, age, city)

# positional -- matched by ORDER; city uses default
student("Asha", 20)
# keyword -- matched by NAME, order doesn't matter
student(name="Ravi", age=21)
# keyword args can be given in ANY order
student(age=19, name="Priya")
student("Kiran", 22, "Chennai")          # override the default
```

| Type                | How it's matched                                       | Example                      |
|---------------------|--------------------------------------------------------|------------------------------|
| Positional argument | By position/order in the call                          | student("Asha", 20)          |
| Keyword argument    | By explicit parameter name                             | student(name="Asha", age=20) |
| Default argument    | Used automatically when the caller omits that argument | def f(x, y=10):              |

### COMMON MISTAKES

- Placing a parameter WITHOUT a default AFTER one WITH a default: `def f(x=5, y):` is a `SyntaxError`. Defaults must come last.
- Using a MUTABLE default argument like a list: `def f(items=[]):`. The same list object is reused across EVERY call that doesn't supply its own list, so items accumulated in one call can leak into the next call. The safe pattern is `def f(items=None):` then `if items is None: items = []` inside the function.

### COMPETITIVE EXAM / GATE FOCUS

The mutable default argument trap is one of the most famous "gotchas" in Python and a favourite interview question: "What's wrong with `def f(x=[]): x.append(1); return x`, called three times in a row?" Answer: the list keeps growing across calls -- `[1]`, then `[1,1]`, then `[1,1,1]` -- because the default list is created ONCE, when the function is defined, not fresh on every call.

## 3. The return Statement

return sends a value back to the caller and immediately ends the function — any code after a return statement inside the same branch never runs. A function with no return statement (or a bare return) automatically returns None.

```
def add(a, b):  
    return a + b          # sends the sum back to the caller  
  
result = add(3, 4)        # result now holds 7  
print(result)  
  
def show(a, b):           # no return statement -- just prints  
    print(a + b)  
  
# prints 7, but x is None, because show() never returned a value  
x = show(3, 4)
```

## Multiple Return Values

Python lets a function return several values at once, packed together as a tuple.

```
def min_max(nums):  
    # actually returns the TUPLE (min(nums), max(nums))  
    return min(nums), max(nums)  
  
# unpacked into two separate variables  
lo, hi = min_max([4, 1, 9, 2])  
print(lo, hi)                # 1 9
```

### COMMON MISTAKES

- Forgetting to capture a function's return value: calling `add(3, 4)` alone (not printing or storing it) computes the sum but throws it away.
- Confusing `print()` (only **DISPLAYS** a value, produces `None`) with `return` (**SENDS** a value back so it can be used later in the program). A function that only prints its answer cannot have that answer used in a further calculation.

## INTERVIEW QUESTIONS

**Q:** What is the difference between `print()` and `return` inside a function?

**A:** `print()` only displays text on the screen and the function's actual return value stays `None` unless `return` is also used. `return` sends a value back to the caller so it can be stored in a variable and reused in further computation. A function used only for its printed output cannot have its result used elsewhere in the program.

## 4. Local vs Global Variables and Scope

**Scope** is the region of a program where a variable name is recognised and usable. A **local variable** is created inside a function and exists only while that function is running; it disappears once the function returns. A **global variable** is created outside all functions, at the top level of the script, and can be READ from inside any function.

```
x = 10                # global variable

def show():
    # local variable -- only exists inside show()
    y = 5
    # can READ the global x, and use its own local y
    print(x, y)

show()                # 10 5
# NameError: y is not defined here -- y was local to show()
print(y)
```

### Modifying a global variable from inside a function

```
count = 0

def increment():
    # UnboundLocalError! Python sees the assignment and treats count as LOCAL
    count += 1

def increment_fixed():
    # tell Python: use the GLOBAL count, don't create a local one
    global count
    count += 1

increment_fixed()
```

```
print(count)          # 1
```

### COMPETITIVE EXAM / GATE FOCUS

This is a genuinely tricky rule and a favourite exam trap: as soon as Python sees an assignment to a name ANYWHERE inside a function body, it treats that name as local for the ENTIRE function, even on lines before the assignment. So merely reading `count` on one line and assigning to it on the next (without the `global` keyword) raises `UnboundLocalError`, because Python already decided `count` is local the moment it saw `count += 1` anywhere in the function.

### MUST REMEMBER

- Local variables live and die inside their function; global variables live at the script's top level.
- A function can READ a global variable freely without any special keyword.
- To ASSIGN to (modify) a global variable inside a function, you must declare ``global name`` first.
- Prefer passing values in as parameters and getting results out via `return`, rather than relying heavily on global variables -- this keeps functions predictable and easier to test.

## 5. Lambda Functions

A **lambda** is a small, unnamed ("anonymous") function, written in a single line, useful for short throwaway logic — especially as an argument to another function.

```
# equivalent to: def square(x): return x * x
square = lambda x: x * x
print(square(5))          # 25
```

```
add = lambda a, b: a + b
print(add(3, 4))          # 7
```

Lambda functions are most useful together with **higher-order functions** — functions that take another function as an argument.

```
nums = [5, 2, 8, 1, 9]
```

```
# apply a function to every item
squared = list(map(lambda x: x * x, nums))
# [25, 4, 64, 1, 81]
print(squared)

# keep only items that pass a test
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)                                # [2, 8]

names = ["Asha", "bob", "Chirag"]
# sort using a custom rule
names.sort(key=lambda s: s.lower())
# ['Asha', 'bob', 'Chirag']
print(names)
```

| Function                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Purpose                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| map(func, iterable)                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Applies func to every item, returns a map object (wrap in list() to see it) |
| filter(func, iterable)                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Keeps only items where func(item) is True                                   |
| sorted(iterable, key=func)                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Sorts using func(item) as the sort key instead of the item itself           |
| <b>COMMON MISTAKES</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                             |
| <ul style="list-style-type: none"> <li>• Writing a lambda with a full statement inside, e.g. <code>lambda x: print(x); return x</code> — a lambda body must be a single <b>EXPRESSION</b>, not a statement; it cannot contain <code>return</code>, assignments, or multiple lines.</li> <li>• Overusing lambdas for anything non-trivial, which hurts readability -- if the logic is more than a short one-liner, a regular <code>def</code> function is almost always clearer.</li> </ul> |                                                                             |

## 6. Modules and Packages

A **module** is simply a `.py` file containing Python code (functions, variables, classes) that can be reused in other programs via `import`. A **package** is a folder containing multiple related modules, organised together.

```
import math                                # import the whole module
# 4.0 -- access with module_name.function_name
print(math.sqrt(16))
```

```

from math import sqrt, pi      # import specific names directly
print(sqrt(16), pi)           # 4.0 3.141592653589793

import math as m               # import with a shorter alias
print(m.sqrt(16))

```

| Form                    | When to use                                                                 |
|-------------------------|-----------------------------------------------------------------------------|
| import module           | Safest -- keeps names namespaced as module.name, avoids clashes.            |
| from module import name | Convenient for a few specific names you'll use often.                       |
| from module import *    | Avoid in real programs -- imports everything, can silently overwrite names. |
| import module as alias  | Shortens a long module name, e.g. import numpy as np.                       |

## Standard Library Modules You Should Know

| Module      | Common use                                                      |
|-------------|-----------------------------------------------------------------|
| math        | sqrt(), floor(), ceil(), pi, factorial()                        |
| random      | randint(), choice(), shuffle(), random()                        |
| datetime    | Working with dates and times                                    |
| os          | Interacting with the operating system / file paths              |
| sys         | Command-line arguments, exiting a program                       |
| collections | Counter, defaultdict -- very useful for competitive programming |

## Creating Your Own Module

```

# --- File: mymath.py ---
def cube(x):
    return x ** 3

# --- File: main.py (in the same folder) ---
import mymath
print(mymath.cube(3))      # 27

```

**if \_\_name\_\_ == "\_\_main\_\_":**

Every Python file has a built-in variable called `__name__`. When you run a file DIRECTLY, Python sets `__name__` to the string `"__main__"`. But when that same file is IMPORTED by another file, `__name__` is set to the module's own name instead. This lets

a file contain code that only runs when the file is executed directly, not when it's imported.

```
# --- File: mymath.py ---
def cube(x):
    return x ** 3

if __name__ == "__main__":
    # only runs when mymath.py is run directly
    print("Testing:", cube(3))
# does NOT run when imported elsewhere
```

### COMPETITIVE EXAM / GATE FOCUS

This pattern is extremely common in real Python code and in interview questions. It allows a file to double as both a reusable module (when imported) and a standalone runnable script with its own test code (when run directly), without the two uses interfering with each other.

### QUICK REVISION

- Function = named reusable block; parameter = placeholder; argument = actual value passed.
- Defaults must come after non-default parameters; never use a mutable default (like []).
- return sends a value back and ends the function; print() only displays, returns None.
- Assigning to a name anywhere in a function makes it local for the WHOLE function unless declared global.
- lambda arg: expression -- single expression only, often paired with map/filter/sorted.
- import module vs from module import name vs import module as alias.
- if \_\_name\_\_ == "\_\_main\_\_": guards code that should run only when the file is executed directly.

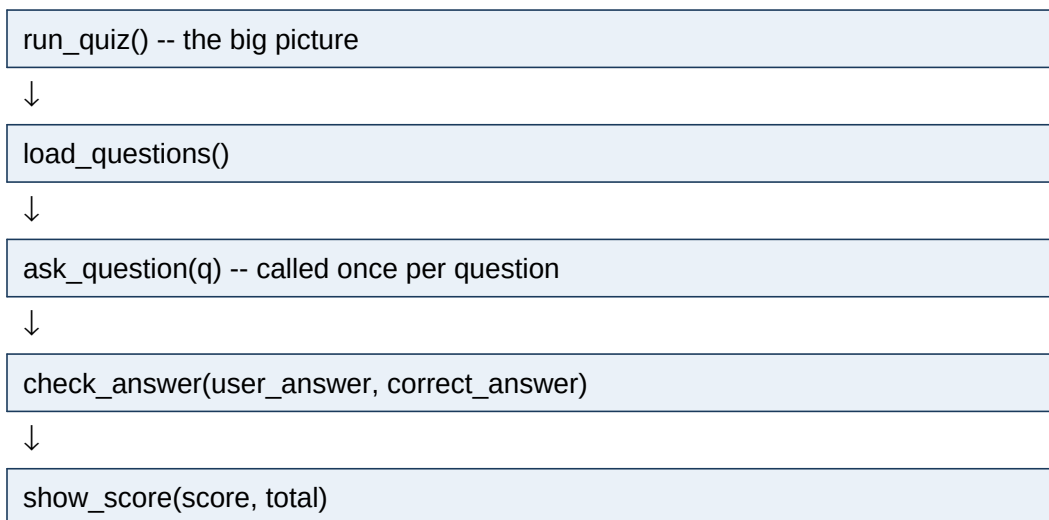
## 7. Function Design and Problem Solving

### Top-Down Design

**Top-down design** means starting with the big picture of a problem and breaking it into smaller and smaller sub-problems, until each piece is simple enough to write directly as a function. This mirrors the Program Development Cycle from Unit I, applied specifically to structuring code with functions.

Example: "Build a simple quiz program." Broken down top-down:

*Top-down breakdown of 'Build a quiz program'*



```
def load_questions():
    return [("2+2?", "4"), ("Capital of France?", "Paris")]

def ask_question(question):
    return input(question + " ")

def check_answer(given, correct):
    return given.strip().lower() == correct.strip().lower()

def show_score(score, total):
    print(f"You scored {score}/{total}")

def run_quiz():
    questions = load_questions()
    score = 0
    for q, correct in questions:
        given = ask_question(q)
```

```
        if check_answer(given, correct):
            score += 1
    show_score(score, len(questions))
```

```
run_quiz()
```

Notice how `run_quiz()` reads almost like a summary of the whole program, because the messy details are hidden inside smaller, well-named functions. This is the entire point of **modular programming**: each function does ONE clear job, can be tested on its own, and can be reused elsewhere.

## Input/Output Separation

A good habit: keep the CALCULATION logic separate from the INPUT and OUTPUT (printing) logic. A function that both takes `input()` and does the calculation is harder to test and reuse, because you can't test the calculation without also providing real keyboard input.

```
# LESS REUSABLE -- input and logic mixed together
```

```
def bmi_bad():
    weight = float(input("Weight (kg): "))
    height = float(input("Height (m): "))
    print(weight / (height ** 2))
```

```
# BETTER -- pure calculation, separate from input/output
```

```
def calculate_bmi(weight, height):
    return weight / (height ** 2)
```

```
def bmi_good():
```

```
    weight = float(input("Weight (kg): "))
    height = float(input("Height (m): "))
    # calculate_bmi can now be tested directly,
    print("BMI:", calculate_bmi(weight, height))
```

```
# e.g. calculate_bmi(70,
```

### COMPETITIVE EXAM / GATE FOCUS

Interviewers specifically look for this separation. A function like `calculate_bmi(weight, height)` that takes plain arguments and returns a plain value is easy to test with fixed sample inputs; a function that reads from `input()` internally cannot be tested this way at all.

## 8. Recursion

### LEVEL 1 — What is recursion?

**Recursion** is when a function calls ITSELF to solve a smaller version of the same problem. Every recursive function needs exactly two parts:

| Part           | Meaning                                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------------------------|
| Base case      | The simplest version of the problem, answered directly WITHOUT another recursive call. This is what stops the recursion. |
| Recursive case | The function calls itself with a SMALLER or SIMPLER version of the input, moving closer to the base case.                |

Think of Russian nesting dolls. To "open all dolls", you open the outer doll, and then your task becomes "open all dolls" again, just on a smaller doll inside. You keep doing the same action (recursive case) until you reach the smallest doll that doesn't open further (base case).

#### COMMON MISTAKES

- Forgetting the base case entirely, or writing a base case that can never actually be reached — both cause the function to call itself forever, eventually crashing with a **RecursionError: maximum recursion depth exceeded** (this is Python's version of a stack overflow).

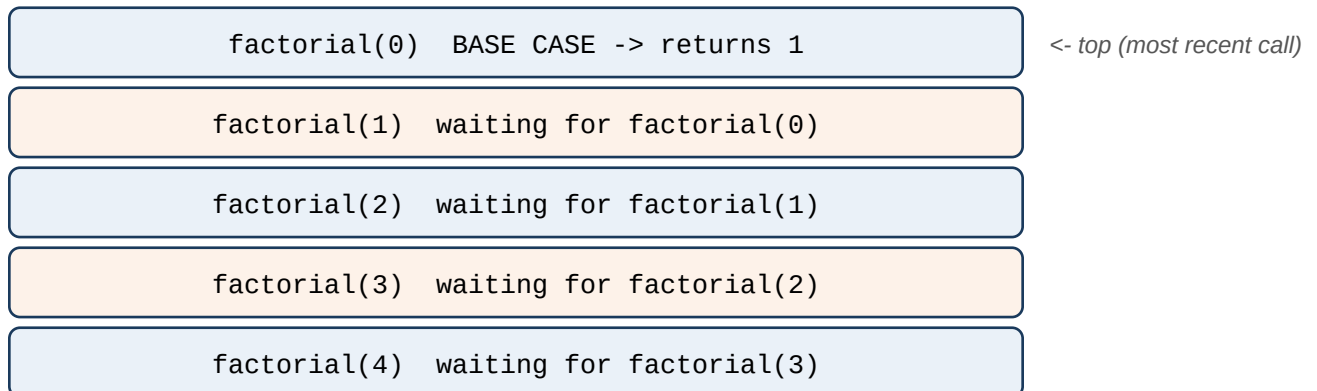
### LEVEL 2 — Factorial: a first full example

Mathematically:  $4!$  ("4 factorial") =  $4 \times 3 \times 2 \times 1 = 24$ . Notice that  $4! = 4 \times 3!$ , and  $3! = 3 \times 2!$ , and so on — each factorial is defined in terms of a SMALLER factorial. The base case is  $0! = 1$  (by mathematical definition).

```
def factorial(n):
    if n == 0:                # BASE CASE -- stops the recursion
        return 1
    # RECURSIVE CASE -- smaller problem: factorial(n-1)
    return n * factorial(n - 1)
```

Dry run for `factorial(4)` — the call stack GROWS as calls are made, then SHRINKS as each call finally returns a value:

Call stack at its **DEEPEST** point (just before unwinding begins)



Each frame is 'frozen', waiting for the call above it to finish and return a value.

## Unwinding: how the return values flow back down

| Step | Code executed           | Variable state after                      |
|------|-------------------------|-------------------------------------------|
| 1    | factorial(0) returns 1  | factorial(1) can now compute $1 * 1 = 1$  |
| 2    | factorial(1) returns 1  | factorial(2) can now compute $2 * 1 = 2$  |
| 3    | factorial(2) returns 2  | factorial(3) can now compute $3 * 2 = 6$  |
| 4    | factorial(3) returns 6  | factorial(4) can now compute $4 * 6 = 24$ |
| 5    | factorial(4) returns 24 | This is the final answer                  |

### COMPETITIVE EXAM / GATE FOCUS

This 'grow then shrink' shape (calls piling up, then unwinding with computed values) is exactly how the call stack behaves for ANY recursive function, and understanding this diagram is the single most important skill for reading, writing, tracing, and debugging recursive code in exams and interviews.

## LEVEL 2-3 — Fibonacci: recursion with TWO recursive calls

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, ... each number is the sum of the two before it.  $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ , with base cases  $\text{fib}(0) = 0$  and  $\text{fib}(1) = 1$ .

```

def fib(n):
    if n == 0:
        return 0          # base case 1
  
```

```

if n == 1:
    return 1                # base case 2
return fib(n - 1) + fib(n - 2)    # TWO recursive calls

```

### COMPETITIVE EXAM / GATE FOCUS

Because `fib()` calls itself TWICE per call (not once, like factorial), the number of calls grows exponentially: computing `fib(n)` this way takes  $O(2^n)$  time, extremely slow for even moderately large `n` (`fib(40)` alone makes over a billion calls). This inefficiency, and how to fix it using memoization (caching already-computed results), is one of the most common recursion interview questions.

```

# Optimised using memoization (caching results already computed)
def fib_memo(n, cache={}):
    if n in cache:
        return cache[n]
    if n <= 1:
        return n
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]
# Time complexity:  $O(2^n)$  ->  $O(n)$  -- each fib(k) computed only once.

```

### LEVEL 2 — Sum of Numbers (a simple, single-recursive-call example)

```

def sum_to_n(n):
    if n == 0:                # base case
        return 0
    return n + sum_to_n(n - 1) # recursive case

```

This has exactly the same shape as `factorial()`, just with `+` instead of `*`. Once you can trace `factorial()`, you can trace this one the same way.

### LEVEL 3 — A Reverse Problem: Reversing a String Recursively

```

def reverse_str(s):
    # base case -- empty or single character
    if len(s) <= 1:
        return s
    # reverse everything AFTER the first char, then append first char
    return reverse_str(s[1:]) + s[0]

```

**Trace: reverse\_str("abc")**

| Step | Code executed      | Variable state after                |
|------|--------------------|-------------------------------------|
| 1    | reverse_str("abc") | calls reverse_str("bc") + "a"       |
| 2    | reverse_str("bc")  | calls reverse_str("c") + "b"        |
| 3    | reverse_str("c")   | calls reverse_str("") + "c"         |
| 4    | reverse_str("")    | base case: returns "" (length <= 1) |
| 5    | unwind: "" + "c"   | returns "c"                         |
| 6    | unwind: "c" + "b"  | returns "cb"                        |
| 7    | unwind: "cb" + "a" | returns "cba" -- final answer       |

**LEVEL 4 — Recursion vs Iteration**

| Recursion                                                                                          | Iteration (loops)                                              |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Function calls itself with a smaller problem                                                       | A loop repeats a block of code                                 |
| Uses the call stack -- each call uses extra memory                                                 | Uses a fixed, small amount of memory (usually)                 |
| Can be more elegant/readable for naturally recursive problems (tree traversal, divide-and-conquer) | Usually faster and more memory-efficient for simple repetition |
| Risk: RecursionError / stack overflow for very deep recursion                                      | No risk of stack overflow                                      |
| Every recursive solution CAN be rewritten as an iterative one (and vice versa)                     | Every iterative solution CAN be rewritten recursively          |

**LEVEL 4 — Advantages, Disadvantages, and Stack Overflow**

- **Advantage:** often much shorter and closer to the mathematical definition of a problem (factorial, Fibonacci, tree/graph traversal).
- **Advantage:** naturally suited to problems that are recursive BY NATURE, like exploring nested folders, or divide-and-conquer algorithms like merge sort.
- **Disadvantage:** uses more memory than iteration, since every call adds a new stack frame.
- **Disadvantage:** can hit Python's recursion limit (by default 1000 calls deep) and raise a RecursionError, sometimes informally called a 'stack overflow', even for a

problem a loop could solve instantly.

**Stack overflow** happens when recursive calls pile up faster than they resolve, usually because the base case is missing, unreachable, or the input isn't actually shrinking toward it.

```
def broken(n):
    return broken(n - 1) + 1    # NO base case -- this NEVER stops
```

```
broken(5)    # RecursionError: maximum recursion depth exceeded
```

## LEVEL 5 — Time Complexity, Space Complexity, Recurrence Relations

The time taken by a recursive function is often expressed as a **recurrence relation** — an equation describing the function's cost in terms of a SMALLER call to itself.

| Function                 | Recurrence relation                           | Time complexity | Space complexity (call stack) |
|--------------------------|-----------------------------------------------|-----------------|-------------------------------|
| factorial(n)             | $T(n) = T(n-1) + O(1)$                        | $O(n)$          | $O(n)$ -- n stacked frames    |
| fib(n) (plain recursion) | $T(n) = T(n-1) + T(n-2) + O(1)$               | $O(2^n)$        | $O(n)$ -- deepest branch only |
| fib_memo(n)              | $T(n) = T(n-1) + O(1)$ (each n computed once) | $O(n)$          | $O(n)$                        |
| sum_to_n(n)              | $T(n) = T(n-1) + O(1)$                        | $O(n)$          | $O(n)$                        |

### COMPETITIVE EXAM / GATE FOCUS

GATE and university exams often give you a recurrence relation and ask you to identify the time complexity, or vice versa. The pattern  $T(n) = T(n-1) + O(1)$  always resolves to  $O(n)$  (you make n calls, each doing constant extra work).  $T(n) = 2T(n-1) + O(1)$  resolves to  $O(2^n)$ .  $T(n) = T(n/2) + O(1)$ , seen in binary search, resolves to  $O(\log n)$ .

## LEVEL 4 — When to Use Recursion, and When to Avoid It

| Use recursion when...                                                                                                                                                | Prefer iteration when...                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| The problem is naturally defined in terms of smaller versions of itself (factorial, tree/graph traversal, divide-and-conquer algorithms like merge sort/quick sort). | The task is simple repetition over a range or collection -- a for/while loop is simpler, faster, and uses far less memory. |

| Use recursion when...                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Prefer iteration when...                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Code clarity matters more than the small performance cost of extra function calls.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | The input size could be very large, risking RecursionError. |
| MUST REMEMBER                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                             |
| <ul style="list-style-type: none"> <li>• Every recursive function needs a base case (stops recursion) and a recursive case (moves toward it).</li> <li>• The call stack grows with each recursive call and unwinds (returns values) once base cases are hit.</li> <li>• <math>T(n) = T(n-1) + O(1) \rightarrow O(n)</math>. <math>T(n) = 2T(n-1) + O(1) \rightarrow O(2^n)</math>. <math>T(n) = T(n/2) + O(1) \rightarrow O(\log n)</math>.</li> <li>• Missing/unreachable base case <math>\rightarrow</math> infinite recursion <math>\rightarrow</math> RecursionError (Python's stack overflow).</li> <li>• Every recursive solution can be rewritten iteratively, and vice versa; choose based on clarity, memory constraints, and whether the problem is naturally recursive.</li> </ul> |                                                             |

## INTERVIEW QUESTIONS

**Q:** What happens in memory when a recursive function is called repeatedly?

**A:** Each call creates a new stack frame holding that call's local variables and parameters, pushed onto the call stack. The frame stays there, paused, until its recursive call returns a value; then it resumes, computes its own result, returns it to whoever called it, and its frame is popped off the stack.

**Q:** Why is plain recursive Fibonacci so slow, and how do you fix it?

**A:** Because  $\text{fib}(n)$  calls  $\text{fib}(n-1)$  and  $\text{fib}(n-2)$ , and those calls overlap heavily -- the same smaller  $\text{fib}(k)$  gets recomputed many times, giving  $O(2^n)$  time. Memoization (caching each  $\text{fib}(k)$  the first time it's computed, in a dict or array) avoids the repeated work, bringing it down to  $O(n)$ .

## Practice Problems (Unit III)

### A. Beginner

#### F1. Function to Sum Two Numbers (syllabus reference)

##### TRY YOURSELF

##### F1. Function to Sum Two Numbers (syllabus reference)

Write a function that takes two numbers as parameters and returns their sum.

**Input:** 5, 7

**Output:** 12

##### HINT

A single-line function body using return.

##### SOLUTION

See code and explanation below.

```
def add(a, b):  
    return a + b
```

```
print(add(5, 7))
```

This is the simplest possible function shape: two parameters in, one value out via return.  $O(1)$  time and space.

#### F2. Factorial Using Recursion (syllabus: GeeksforGeeks factorial-of-number-using-recursion)

**TRY YOURSELF****F2. Factorial Using Recursion (syllabus: GeeksforGeeks factorial-of-number-using-recursion)**

Write a recursive function to compute  $n!$  for a given  $n$ .

**Input:** 5

**Output:** 120

**HINT**

Base case:  $0! = 1$ . Recursive case:  $n! = n * (n-1)!$ .

**SOLUTION**

See code and explanation below.

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

```
print(factorial(5))
```

See the full call-stack trace earlier in this chapter for exactly how this unwinds.  $O(n)$  time,  $O(n)$  space (call stack depth).

**B. Easy****F3. Two Sum (syllabus: LeetCode two-sum)**

**TRY YOURSELF****F3. Two Sum (syllabus: LeetCode two-sum)**

Given a list of numbers and a target, return the indices of the two numbers that add up to target.

**Input:** nums=[2,7,11,15], target=9

**Output:** [0, 1]

**HINT**

Use a dictionary to remember each number's index as you scan; for each number, check if (target - number) has already been seen.

**SOLUTION**

See code and explanation below.

```
def two_sum(nums, target):
    seen = {}                # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2, 7, 11, 15], 9))
```

This function wraps the hash-map pattern from Unit II inside a clean, reusable function with a clear return value -- a good example of separating logic from input/output. Brute force nested loops would be  $O(n^2)$ ; this hash-map approach is  $O(n)$ .

**F4. Valid Anagram (syllabus: LeetCode valid-anagram)**

**TRY YOURSELF****F4. Valid Anagram (syllabus: LeetCode valid-anagram)**

Given two strings, return True if one is an anagram of the other (same letters, same counts, any order).

**Input:** s1="listen", s2="silent"

**Output:** True

**HINT**

Two strings are anagrams if their sorted characters are identical, OR if their character frequency dictionaries match.

**SOLUTION**

See code and explanation below.

```
def is_anagram(s1, s2):  
    if len(s1) != len(s2):  
        return False  
    return sorted(s1) == sorted(s2)  
  
print(is_anagram("listen", "silent"))
```

Sorting both strings and comparing is a clean, short solution:  $O(n \log n)$  time. A frequency-dictionary approach (build freq counts for both strings using the Unit II pattern, then compare the two dictionaries) achieves  $O(n)$  time instead, and is the preferred answer in interviews focused on optimal complexity.

## C. Medium / Competitive

**F5. Compute the Average (syllabus: CodeChef FLOW002, as a function)**

**TRY YOURSELF****F5. Compute the Average (syllabus: CodeChef FLOW002, as a function)**

Write a function that takes a list of numbers and returns their average, separated from any input/output code.

**Input:** [10, 20, 30]

**Output:** 20.0

**HINT**

Use `sum()` and `len()`, and keep this function pure -- no `input()`/`print()` inside it.

**SOLUTION**

See code and explanation below.

```
def average(nums):  
    return sum(nums) / len(nums)  
  
def average_bad(nums):  
    # BAD: no return value, cannot be reused in other calculations  
    print(sum(nums) / len(nums))  
  
print(average([10, 20, 30]))
```

This problem is really about the input/output separation principle from this chapter: `average()` returns a usable value, so it can be plugged into further calculations, tested directly with fixed sample lists, or reused inside a bigger program.  $O(n)$  time.

**F6. Power Function Using Recursion (original problem, same style as factorial)**

**TRY YOURSELF****F6. Power Function Using Recursion (original problem, same style as factorial)**

Write a recursive function `power(base, exp)` that computes base raised to `exp` (a non-negative integer), without using `**`.

**Input:** `base=2, exp=5`

**Output:** 32

**HINT**

Base case: `exp == 0` returns 1. Recursive case: `power(base, exp) = base * power(base, exp-1)`.

**SOLUTION**

See code and explanation below.

```
def power(base, exp):
    if exp == 0:                # base case
        return 1
    return base * power(base, exp - 1)  # recursive case

print(power(2, 5))
```

Same shape as `factorial()`: one base case, one recursive call reducing the problem by 1 each time.  $O(\text{exp})$  time and space. (A faster  $O(\log \text{exp})$  version exists using the idea `power(b,e) = power(b,e//2)**2`, adjusted for odd `e` -- worth knowing for the Competitive Extension.)

## D. Interview-style

**F7. Check if a Number is a Power of Two, Recursively**

**TRY YOURSELF****F7. Check if a Number is a Power of Two, Recursively**

Write a recursive function to check whether a positive integer  $n$  is a power of two (1, 2, 4, 8, 16, ...).

**Input:** 16

**Output:** True

**HINT**

Base cases:  $n == 1 \rightarrow \text{True}$ ;  $n$  is odd and not 1  $\rightarrow \text{False}$ . Recursive case: if  $n$  is even, check  $n // 2$ .

**SOLUTION**

See code and explanation below.

```
def is_power_of_two(n):
    if n == 1:
        return True
    if n % 2 != 0 or n == 0:
        return False
    return is_power_of_two(n // 2)

print(is_power_of_two(16))
# False -- 18 is even but not a power of two
print(is_power_of_two(18))
```

Every recursive call halves  $n$ , so the recursion depth is  $O(\log n)$  -- notice how this matches the  $T(n) = T(n/2) + O(1)$  recurrence pattern from the table earlier in this chapter. This 'keep halving until you hit a base case' shape is extremely common in interview recursion questions.

## Master Comparison Tables (Unit III)

### Parameter vs Argument

| Parameter                                            | Argument                                            |
|------------------------------------------------------|-----------------------------------------------------|
| Placeholder name in the function DEFINITION          | Actual value passed in the function CALL            |
| <code>def greet(name):</code> -- name is a parameter | <code>greet("Asha")</code> -- "Asha" is an argument |

### Local vs Global Variable

| Local variable                                          | Global variable                                                      |
|---------------------------------------------------------|----------------------------------------------------------------------|
| Created inside a function; exists only during that call | Created at the top level of the script; exists for the whole program |
| Cannot be accessed from outside its function            | Can be READ from inside any function automatically                   |
| Needs no special keyword to create                      | Needs `global name` inside a function to be MODIFIED there           |

### Recursion vs Iteration

| Recursion                                      | Iteration                                          |
|------------------------------------------------|----------------------------------------------------|
| Function calls itself; uses the call stack     | Loop repeats; uses (usually) constant extra memory |
| Risk of RecursionError for very deep problems  | No stack-depth risk                                |
| Often clearer for naturally recursive problems | Often faster and more memory-efficient generally   |

## Unit III Mock Test

1. `def f(x, y=5): return x+y` -- what does `f(3)` return? (a) Error (b) 3 (c) 8 (d) 5
2. Which is a SyntaxError? (a) `def f(x, y=1):` (b) `def f(x=1, y):` (c) `def f(x, y):` (d) `def f(x=1, y=2):`
3. What does a function with no return statement return? (a) 0 (b) "" (c) None (d) Error

4. `x = 5`  
`def f():`  
`x = 10`  
`print(x)`  
`f()`  
`print(x)` -- output? (a) 10 10 (b) 5 5 (c) 10 5 (d) 5 10
5. What is the base case of `factorial(n)`? (a) `n==1` (b) `n==0`, returns 1 (c) `n<0` (d) there is none
6. `lambda x: x*2` is equivalent to: (a) `def f(x): return x*2` (b) `def f(x): x*2` (c) `def f(): return x*2` (d) None of these
7. What does 'global x' inside a function do? (a) creates a new local x (b) lets the function modify the global x (c) deletes x (d) nothing
8. Time complexity of plain recursive `fib(n)`? (a)  $O(n)$  (b)  $O(\log n)$  (c)  $O(2^n)$  (d)  $O(n^2)$
9. `from math import sqrt` -- correct usage after this line? (a) `math.sqrt(4)` (b) `sqrt(4)` (c) `sqrt.math(4)` (d) `import.sqrt(4)`
10. What causes a `RecursionError`? (a) too many parameters (b) missing/unreachable base case (c) using `return` (d) using `lambda`

## Coding Questions

- Write a function `is_prime(n)` that returns `True/False`, and use it to print all primes from 1 to 50.
- Write a recursive function to compute the sum of digits of a number.
- Write a function that takes a list and returns a NEW list with duplicates removed, without using `set()`.
- Write a recursive function `gcd(a, b)` using the Euclidean algorithm: `gcd(a,b) = gcd(b, a%b)`, base case `gcd(a,0) = a`.
- Trace by hand: `def f(n):\n if n==0: return 0\n return n + f(n-1)\nprint(f(4))` -- draw the call stack and show the unwind, like the factorial example in this chapter.

## Answer Key

| Q | Answer                                                | Q  | Answer                                       |
|---|-------------------------------------------------------|----|----------------------------------------------|
| 1 | (c) 8 -- x=3, y uses default 5                        | 6  | (a) def f(x): return x*2                     |
| 2 | (b) def f(x=1, y): -- default before non-default      | 7  | (b) allows modifying the global x            |
| 3 | (c) None                                              | 8  | (c) $O(2^n)$ -- two recursive calls per call |
| 4 | (c) 10 5 -- x inside f() is a separate local variable | 9  | (b) sqrt(4) -- imported directly by name     |
| 5 | (b) n==0, returns 1 (by mathematical definition)      | 10 | (b) missing/unreachable base case            |

Coding question 5 trace:  $f(4) \rightarrow f(3) \rightarrow f(2) \rightarrow f(1) \rightarrow f(0)$  returns 0 (base case).  
 Unwinding:  $f(1)=1+0=1$ ,  $f(2)=2+1=3$ ,  $f(3)=3+3=6$ ,  $f(4)=4+6=10$ .

*End of Unit III. Unit IV (Object-Oriented Programming and File Operations) will follow next, with the same visual and interactive structure.*